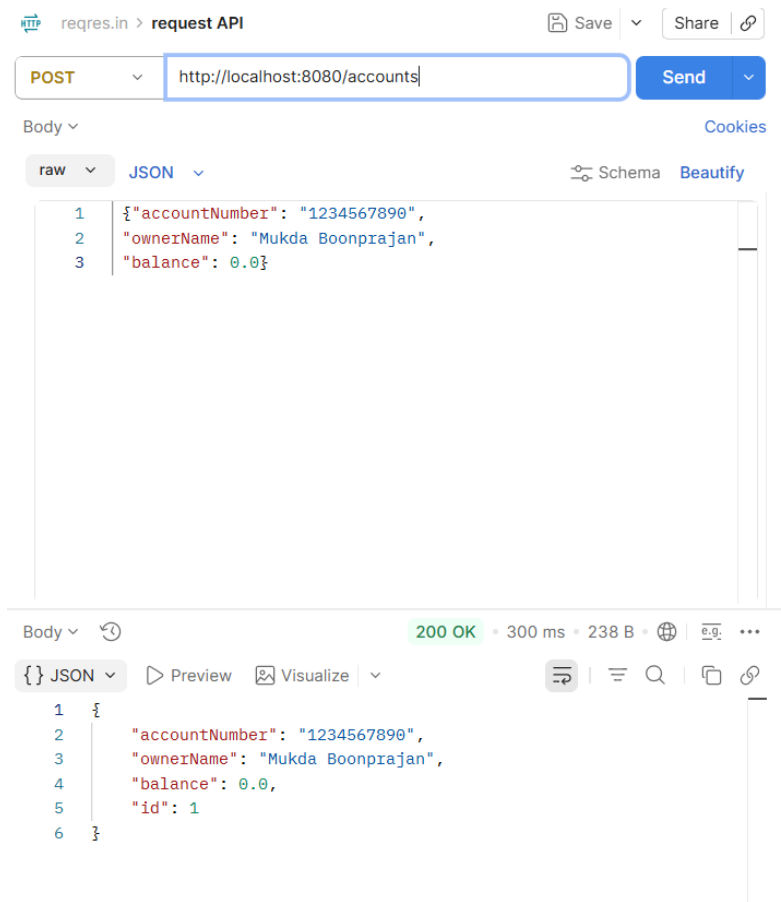


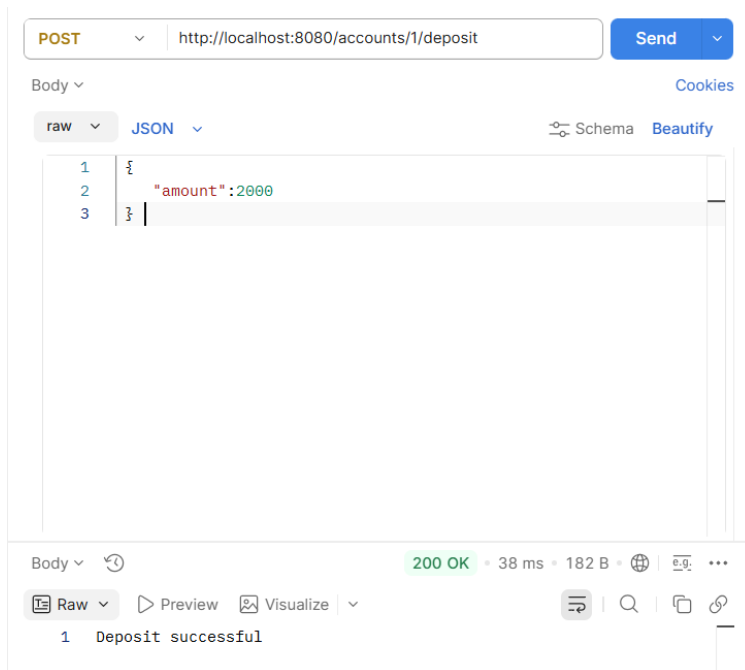
Lab 9

1. การสร้าง Account



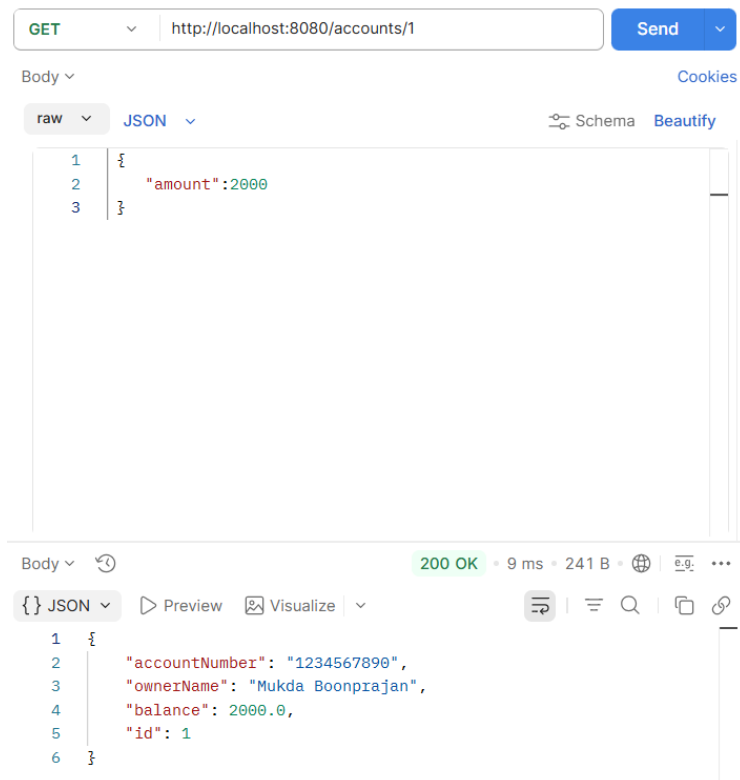
ระบบสามารถสร้าง Account ได้สำเร็จ โดยระบบจะกำหนดค่า ID ให้กับบัญชีโดยอัตโนมัติ และสามารถนำ ID ที่ได้รับไปใช้สำหรับทำรายการฝากเงินในขั้นตอนถัดไป

2.การฝากเงิน



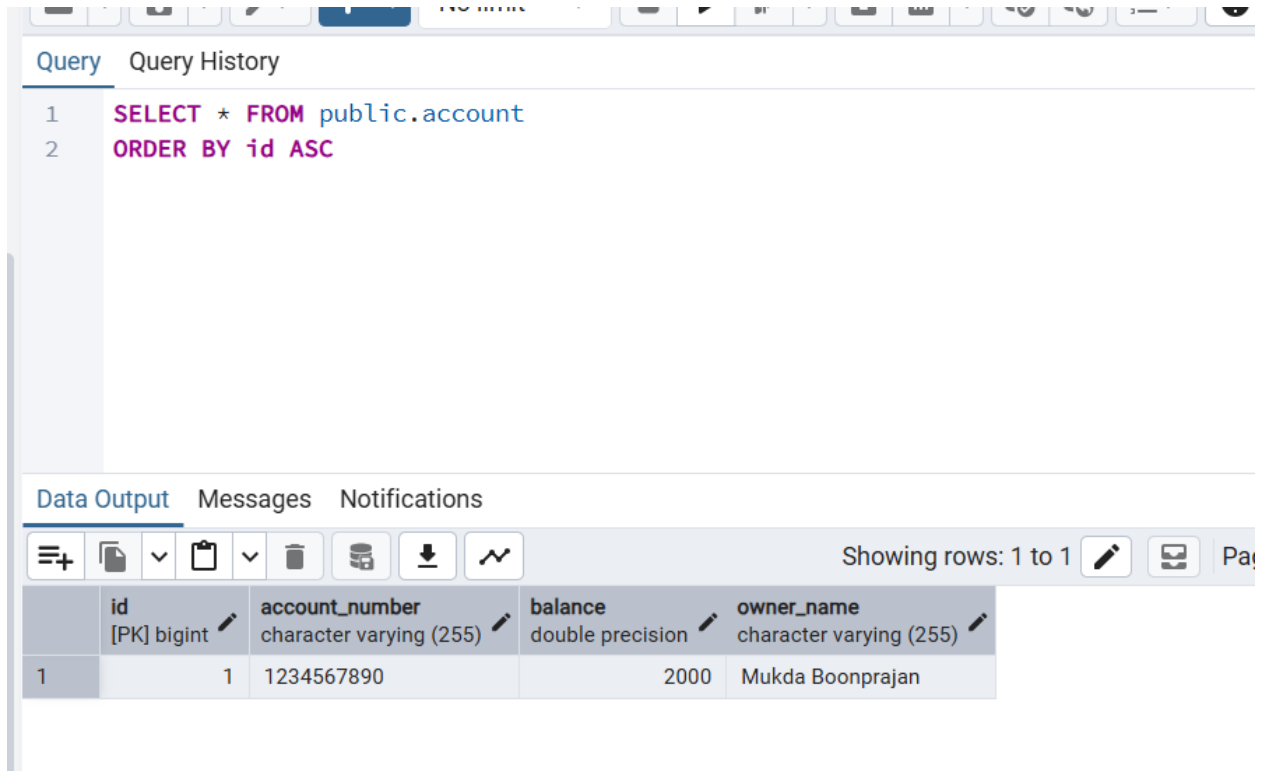
ระบบสามารถดำเนินการฝากเงินได้สำเร็จ โดยยอดเงินใน Account เพิ่มขึ้นจาก 0 บาท เป็น 1,000 บาท พร้อมทั้งมีการบันทึกข้อมูลรายการฝากเงินลงในตาราง `deposit_transaction` เรียบร้อยแล้ว

3.ตรวจสอบ account



จากการตรวจสอบพบว่า ยอดเงินในบัญชีถูกปรับเพิ่มตามจำนวนเงินที่ฝาก เช่น ยอดเงินจากเดิม 0 บาท เปลี่ยนเป็น 1,000 บาท แสดงว่าระบบสามารถอัปเดตยอดเงินใน Account ได้ถูกต้องตามรายการฝากเงิน

4. ตรวจสอบ DepositTransaction



The screenshot shows a database query interface. The 'Query' tab is active, displaying the following SQL query:

```
1 SELECT * FROM public.account
2 ORDER BY id ASC
```

The 'Data Output' tab is also visible, showing the results of the query. The results are displayed in a table with the following columns: id, account_number, balance, and owner_name. The table shows one row of data:

id	account_number	balance	owner_name
1	1234567890	2000	Mukda Boonprajan

จากผลการทดลองพบว่า ระบบสามารถบันทึกรายการฝากเงินลงในฐานข้อมูลได้ โดยข้อมูลที่บันทึกประกอบด้วย id, amount และ account_id ซึ่งเป็นข้อมูลสำคัญของรายการฝากเงิน

5. ทดลอง Rollback



The screenshot shows an API response with a status of 500 Internal Server Error. The response is displayed in the 'Body' tab, showing the following JSON data:

```
1 {
2   "timestamp": "2026-09-11T07:17:33.616+00:00",
3   "status": 500,
4   "error": "Internal Server Error",
5   "path": "/accounts/1/deposit"
6 }
```

@Transactional ทำให้การเพิ่มยอดเงินและการบันทึกประวัติการฝากเงินถูกควบคุมให้อยู่ภายใน Transaction เดียวกัน หากขั้นตอนใดขั้นตอนหนึ่งเกิดข้อผิดพลาด ระบบจะยกเลิกการเปลี่ยนแปลงทั้งหมด

6. ตรวจสอบว่าข้อมูลถูกย้อนกลับไหม

The screenshot shows a Postman interface with a successful HTTP response. The status bar indicates "200 OK" with a response time of 16 ms and a body size of 241 B. The response body is in JSON format, showing a successful deposit transaction.

```
1 {
2   "accountNumber": "1234567890",
3   "ownerName": "Mukda Boonprajan",
4   "balance": 2000.0,
5   "id": 1
6 }
```

Below the JSON, the "Data Output" tab is selected, displaying a table with the transaction details:

	id [PK] bigint	account_number character varying (255)	balance double precision	owner_name character varying (255)
1	1	1234567890	2000	Mukda Boonprajan

ผลที่ถูกต้องยังเป็น 1000 เพราะ Transaction ถูก Rollback และ ตาราง deposit_transaction ยังมี 1 แถวเท่าเดิม Response จาก Postman ที่แสดงว่า balance ไม่เพิ่มและไม่มีการเพิ่มรายการใหม่

7. ทดลองโดยเอา @Transactional ออก

The screenshot shows a Postman interface with an "Internal Server Error" response. The status bar indicates "500 Internal Server Error" with a response time of 423 ms and a body size of 273 B. The response body is in JSON format, showing an error message.

```
1 {
2   "timestamp": "2026-09-11T07:27:55.997+00:00",
3   "status": 500,
4   "error": "Internal Server Error",
5   "path": "/accounts/1/deposit"
6 }
```

ให้ลบ @Transactional ออก โดยยังคงบรรทัด throw new RuntimeException("Test Rollback"); ไว้เหมือนเดิม จากนั้นทดลองฝากเงินอีกครั้ง พบว่าได้รับ 500 Internal Server Error เนื่องจากไม่มี Transaction ครอบคลุมการทำงาน ดังนั้นการเปลี่ยนแปลงข้อมูลที่เกิดขึ้นก่อนเกิด Exception อาจไม่ถูกย้อนกลับ (Rollback)

8. ผลการทดลองที่แสดงความแตกต่าง

8.1เมื่อมี @Transactional

คำสั่งเพิ่มยอดเงินและบันทึกประวัติการฝากเงินจะถูกจัดให้อยู่ใน Transaction เดียวกัน หากเกิดข้อผิดพลาดระหว่างการทำงาน ระบบจะทำการ Rollback การเปลี่ยนแปลงทั้งหมด ทำให้ข้อมูลกลับไปเป็นสถานะก่อนเริ่มฝากเงิน

Data Output Messages Notifications				
Showing rows: 1				
	id [PK] bigint	account_number character varying (255)	balance double precision	owner_name character varying (255)
1	1	1234567890	2000	Mukda Boonprajan

8.2 เมื่อไม่มี @Transactional

การเปลี่ยนแปลงในแต่ละส่วนไม่ได้ถูกควบคุมให้อยู่ใน Transaction เดียวกัน เมื่อเกิดข้อผิดพลาด ข้อมูลบางส่วนอาจถูกบันทึกไปแล้วและไม่ถูกยกเลิก ส่งผลให้ยอดเงินกับประวัติการฝากเงินอาจไม่สอดคล้องกัน

Query Query History	
1	SELECT * FROM public.account
2	ORDER BY id ASC

Data Output Messages Notifications				
Showing rows: 1 to				
	id [PK] bigint	account_number character varying (255)	balance double precision	owner_name character varying (255)
1	1	1234567890	3000	Mukda Boonprajan

9. คำถามท้าย Lab

1. @Transactional มีหน้าที่อะไร?

@Transactional ใช้สำหรับกำหนดขอบเขตการทำงานของ Transaction ภายในเมธอด โดยคำสั่งต่าง ๆ จะถูกดำเนินการเป็นชุดเดียวกัน หากทุกขั้นตอนทำงานสำเร็จ ระบบจะ COMMIT ข้อมูล แต่หากเกิด Exception ขึ้น ระบบจะ ROLLBACK การเปลี่ยนแปลงที่เกิดขึ้นภายใน Transaction

2. เหตุใดการฝากเงินจึงควรใช้ Transaction?

การฝากเงินมีมากกว่าหนึ่งขั้นตอน เช่น การเพิ่มจำนวนเงินใน Account และการสร้างข้อมูลประวัติใน DepositTransaction ซึ่งข้อมูลทั้งสองส่วนควรเปลี่ยนแปลงไปพร้อมกัน หากขั้นตอนใดล้มเหลว ควรยกเลิกการเปลี่ยนแปลงทั้งหมด เพื่อป้องกันปัญหา เช่น ยอดเงินถูกเพิ่มแต่ไม่มีข้อมูลประวัติการฝากเงิน ดังนั้น Transaction จึงช่วยให้กระบวนการทำงานมีความถูกต้อง โดยต้องสำเร็จทั้งหมดหรือยกเลิกทั้งหมด

3. COMMIT และ ROLLBACK ต่างกันอย่างไร?

COMMIT เป็นการยืนยันการเปลี่ยนแปลงที่เกิดขึ้นใน Transaction ทำให้ข้อมูลถูกบันทึกลง Database อย่างถาวร ส่วน ROLLBACK เป็นการย้อนกลับการเปลี่ยนแปลงที่เกิดขึ้นภายใน Transaction ทำให้ข้อมูลกลับสู่สถานะเดิมก่อนเริ่มทำการรายการ

4. Account และ DepositTransaction มีความสัมพันธ์แบบใด?

Account กับ DepositTransaction มีความสัมพันธ์แบบ One-to-Many (1:N) เนื่องจาก Account หนึ่งบัญชีสามารถมีรายการฝากเงินได้หลายรายการ แต่ DepositTransaction หนึ่งรายการจะอ้างอิงกับ Account เพียงหนึ่งบัญชี

5. เมื่อเกิด Error ขณะใช้ @Transactional ข้อมูลใน Database เปลี่ยนแปลงอย่างไร?

หากเกิด Error หรือ Exception ระหว่างที่ Transaction ยังทำงานอยู่ ระบบจะ ROLLBACK การเปลี่ยนแปลงที่เกิดขึ้นทั้งหมดภายใน Transaction ตัวอย่างเช่น หากระบบเพิ่มยอดเงินแล้วเกิด Error ในขั้นตอนถัดไป ยอดเงินและประวัติการฝากเงินจะไม่ถูกบันทึกไว้ ทำให้ Database กลับมาเป็นสถานะก่อนทำการรายการ

6. จากการทดลอง เมื่อลบ @Transactional ออก ผลลัพธ์แตกต่างจากเดิมอย่างไร?

เมื่อเอา @Transactional ออก การทำงานทั้งหมดจะไม่ได้อยู่ภายใต้ Transaction เดียวกัน เมื่อเกิด Error ขึ้นระหว่างกระบวนการ การเปลี่ยนแปลงที่สำเร็จไปก่อนหน้านี้อาจยังคงอยู่ใน Database และไม่ถูก Rollback ทำให้ข้อมูลบางส่วน เช่น ยอดเงินและประวัติการฝากเงิน อาจเกิดความไม่สอดคล้องกัน